# Exercises

## 02. Introduction to Cryptography

# Question 1

<u>You are having a discussion with a friend about cryptography. Your friend makes a series of statements. Please tell us how you would respond (True or False) and motivate your answer.</u>

A. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.
B. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.
C. The reason why the 2048bit RSA is more robust to brute forcing that a 256bit AES, is because the key is longer.

A. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

***[see slides on "cryptography"]*** *False. A perfect cipher, i.e., a one-time pad with a key chosen uniformly at random is not vulnerable to an exhaustive search as all the keys will yield a potentially meaningful plaintext (with the same likelihood of being the good one).*

B. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.

A. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

***[see slides on "cryptography"]*** *False. A perfect cipher, i.e., a one-time pad with a key chosen uniformly at random is not vulnerable to an exhaustive search as all the keys will yield a potentially meaningful plaintext (with the same likelihood of being the good one).*

B. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.

***[see slides on "cryptography"]*** *False. Generally, a cryptosystem is broken if there is any attack faster than trying all the possible keys to obtain the plaintext.*

C. The reason why the 2048bit RSA is more robust to brute forcing that a 256bit AES, is because the key is longer.

A. No encryption algorithm is perfect, as they are all vulnerable to brute forcing.

***[see slides on "cryptography"]*** *False. A perfect cipher, i.e., a one-time pad with a key chosen uniformly at random is not vulnerable to an exhaustive search as all the keys will yield a potentially meaningful plaintext (with the same likelihood of being the good one).*

B. An encryption algorithm is broken when there is at least one way to derive the key from a given amount of ciphertext.

***[see slides on "cryptography"]*** *False. Generally, a cryptosystem is broken if there is any attack faster than trying all the possible keys to obtain the plaintext.*

C. The reason why the 2048bit RSA is more robust to brute forcing that a 256bit AES, is because the key is longer.

***[see slides on "cryptography"]*** *False. The size of the key cannot be used as a <u>direct</u> comparison criterion because the required computational effort depends on the key length in a different way depending on the cipher.*

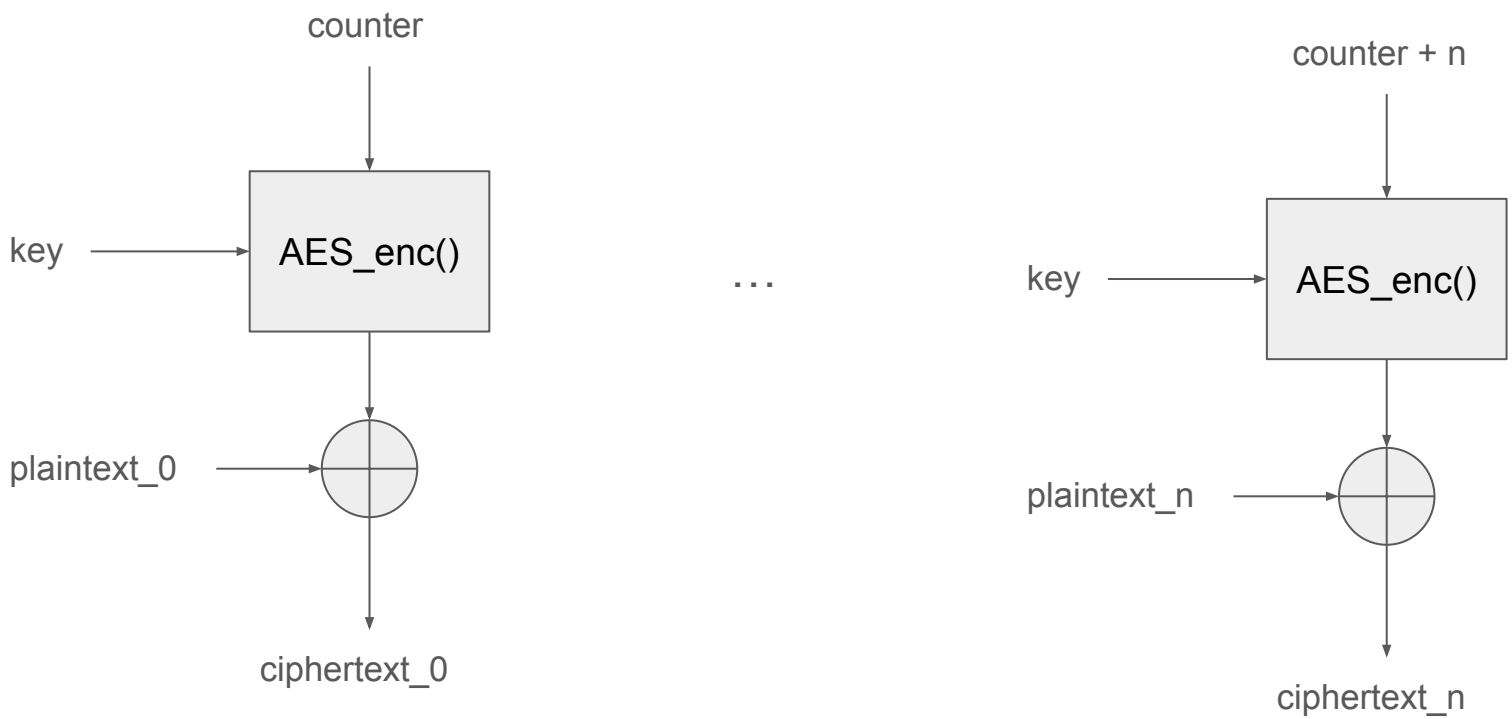
# Question 2

Consider a data protection mechanism which encrypts an entire hard disk, block by block, by means of AES in Counter (CTR) mode, employing a 128 bit key.

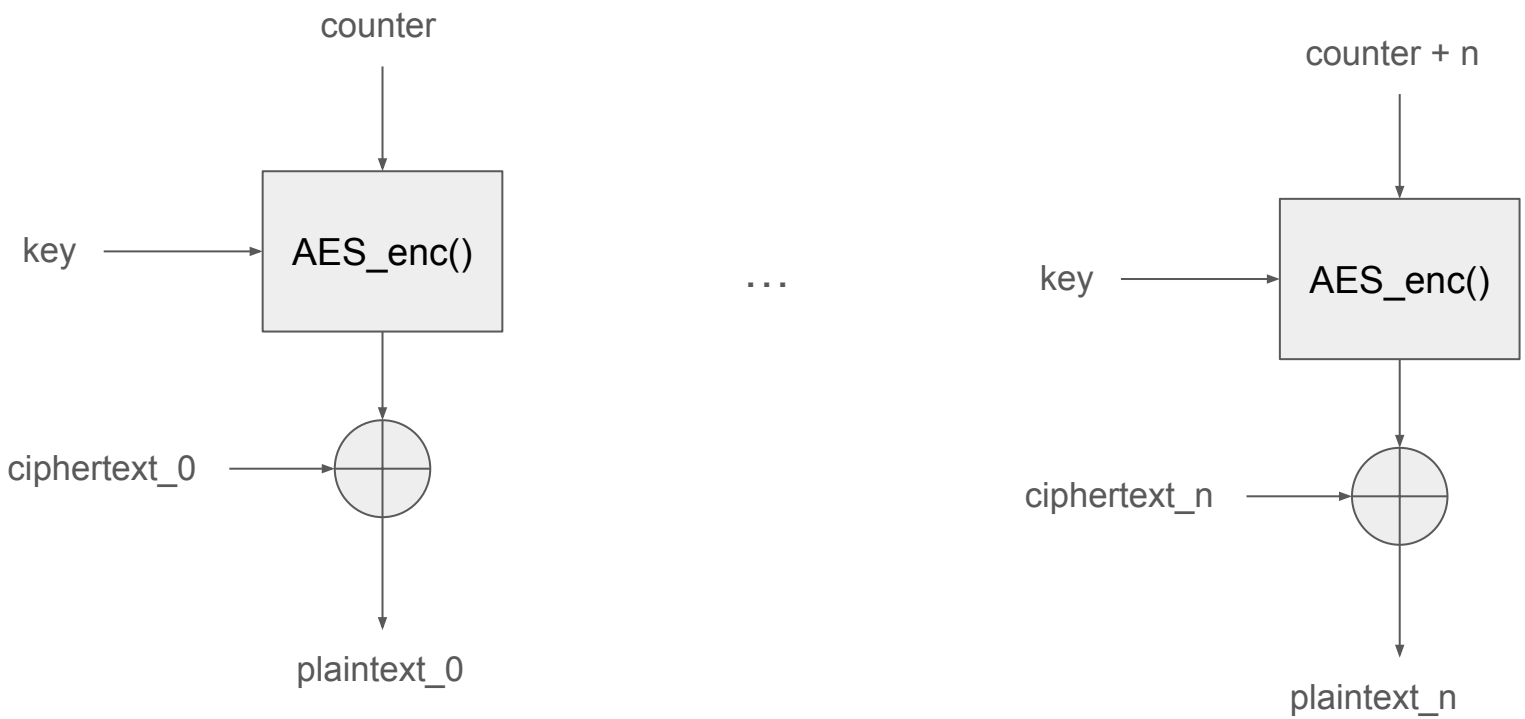
The system administrator, following a new directive which mandates keys to be at least 256 bits long, implements the following compatibility measure: it encrypts the volume again, with the same 128 bit key and counter.

- Argue on whether the method provides a security margin which is larger, smaller or the same with respect to the original encryption scheme.
- Describe an alternative measure to comply with the directory, other than decrypting and re-encrypting the entire volume.

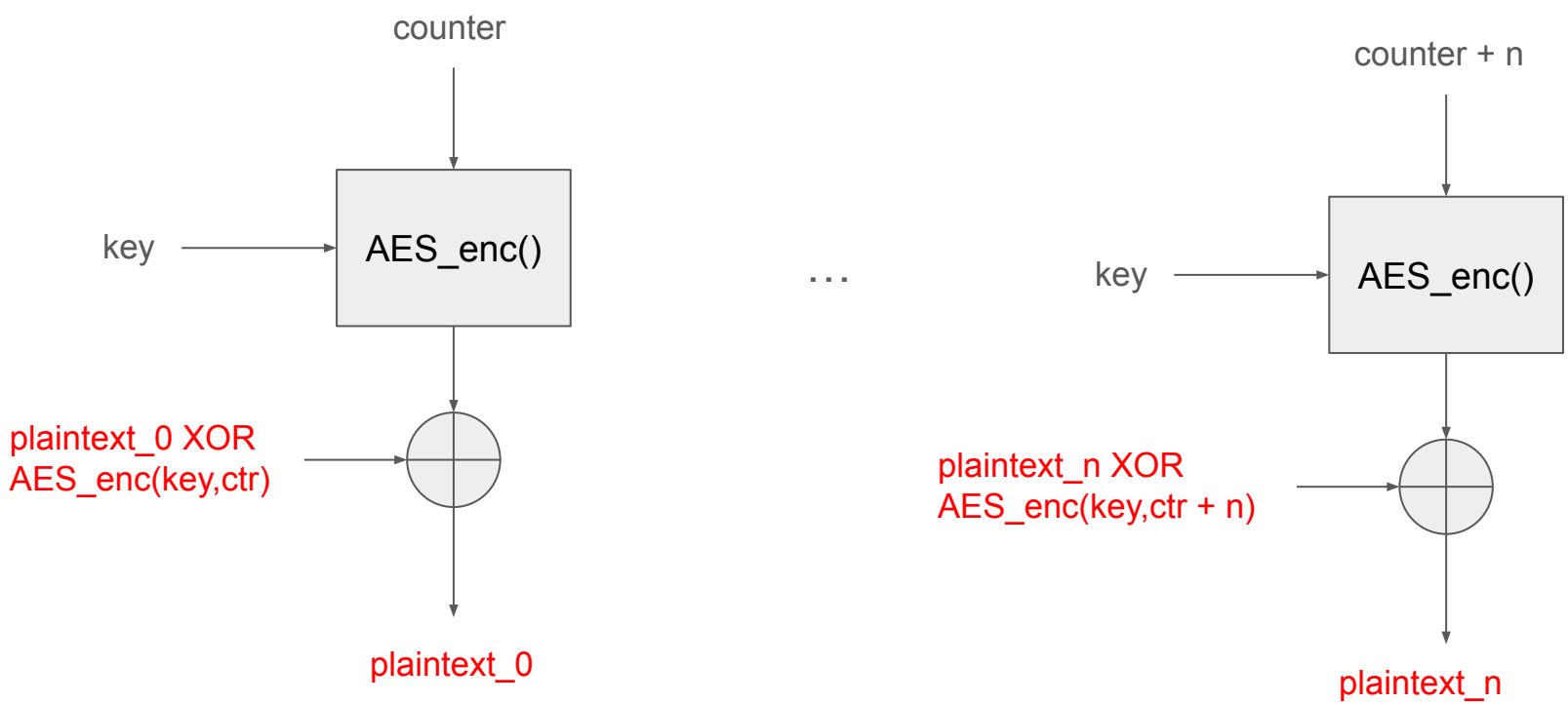
# Recall on AES-CTR - Encryption



# Recall on AES-CTR - Decryption



# Recall on AES-CTR - Security Measure




It is decrypting the original data!

Encrypting with AES-CTR and a different 128 bit key actually solves the decryption issue, and provides 256 bits of equivalent security

(under the largely believed assumption that AES is not a group).

Considering the aforementioned scenario, is it possible to claim that the information on the disk cannot be tampered with in a meaningful way, given that all the information on disk is fully encrypted? Either support the claim or disprove it providing a practical example and a solution to prevent tampering

Considering the aforementioned scenario, is it possible to claim that the information on the disk cannot be tampered with in a meaningful way, given that all the information on disk is fully encrypted? Either support the claim or disprove it providing a practical example and a solution to prevent tampering

Encrypting data with AES in counter mode does not provide any protection against tampering. Indeed, an attacker could modify the ciphertext at her own will, knowing that a bit flip in the ciphertext will result in a bit flip in the plaintext, in the same position.

Adding a message authentication code (MAC) to the data (e.g., disk-block-wise) prevents tampering altogether.

# Question 3

Consider the case of an air-gapped embedded device (i.e., without any network connection or similar communication means), where it is desired to have firmware updates performed connecting a mass storage device to it. The device does not have any permanent energy source (battery or connection to mains), and is thus turned off between uses.

Argue on whether it is possible or not to employ digital certificates and a PKI infrastructure to validate the origin authenticity of the firmware updates.

In case this is possible, sketch briefly the firmware validation procedure, otherwise highlight the reason why it cannot be performed, and provide a solution.

Consider the case of an air-gapped embedded device (i.e., without any network connection or similar communication means), where it is desired to have firmware updates performed connecting a mass storage device to it. The device does not have any permanent energy source (battery or connection to mains), and is thus turned off between uses. Argue on whether it is possible or not to employ digital certificates and a PKI infrastructure to validate the origin authenticity of the firmware updates. In case this is possible, sketch briefly the firmware validation procedure, otherwise highlight the reason why it cannot be performed, and provide a solution.

An air-gapped device without permanent power supply is not able to keep a reference to the passing of time, as a consequence, it is not possible to perform certificate validation due to the inability of testing for certificate expiration.

This shortcoming can either be circumvented switching to an alternate firmware validation approach (e.g., **employing a static public key to perform signature verification**), or obtaining a time reference. A viable approach to limit the connection of the device with the outer world is to decode the MSF-60 radio time reference, or to employ a GPS module. Note that, while at considerable effort, both can be spoofed.

# Exercise 4

A PIN selection form asks the user to pick a 6 decimal digits number to be employed to unlock a smartphone. Considering the case where a certain amount of users pick their birthdate, encoded as YYMMDD (least significant digits of the year, followed by month and day of birth) as their PIN, what is the average amount of attempts you have to make to guess one of their PINs?

Is it larger, smaller or about the same amount of attempts you have to make to guess a random sweep pattern over all the dots in a 2 x 4 grid?

# Exercise 4 - PIN

- There are 365 days in a year
- Let’s assume the birthday is in the last 100 years

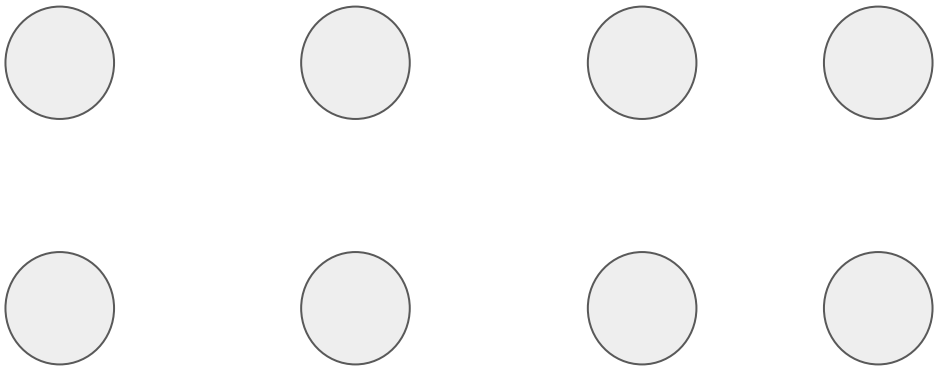
possible_combinations = 365 * 100 = 36.500

- Assume we can ignore the bias in the dates of birth (which are not uniformly distributed over the year)
- About 36.500 guesses will yield to the correct PIN!
- On an average, you can get the correct guess out of the 35.000 random choices in

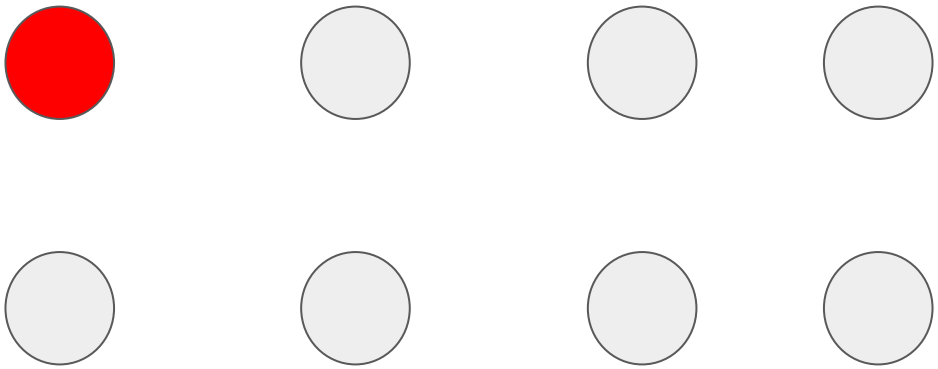
  36.500 / 2

  attempts!

# Exercise 4 - Sweep Pattern

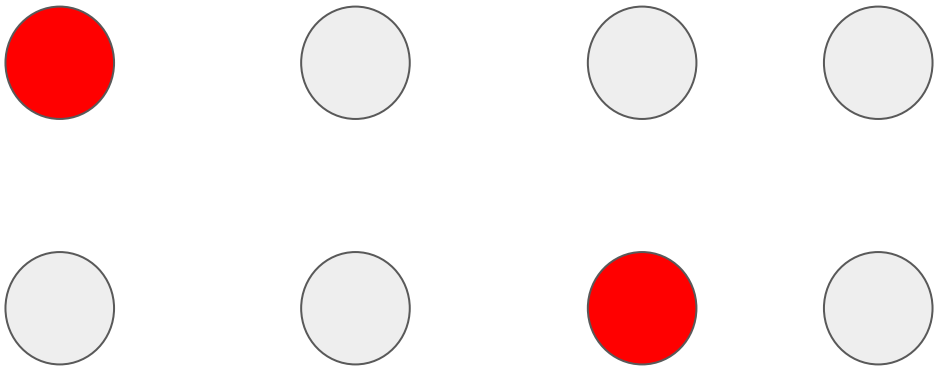


# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of 8

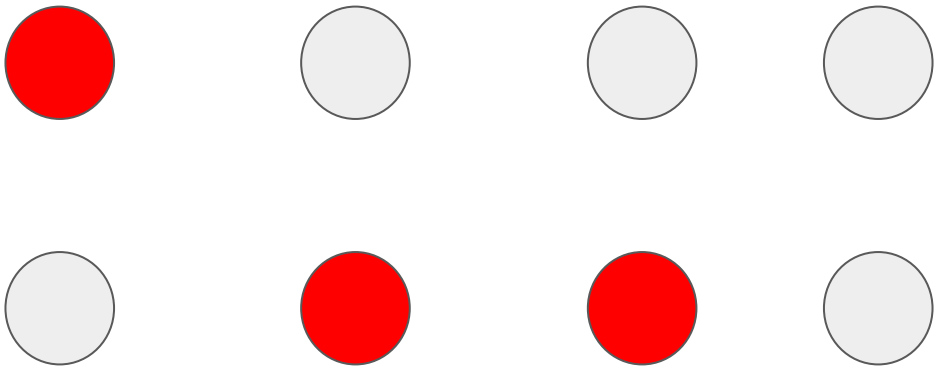
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 7

8 * 7

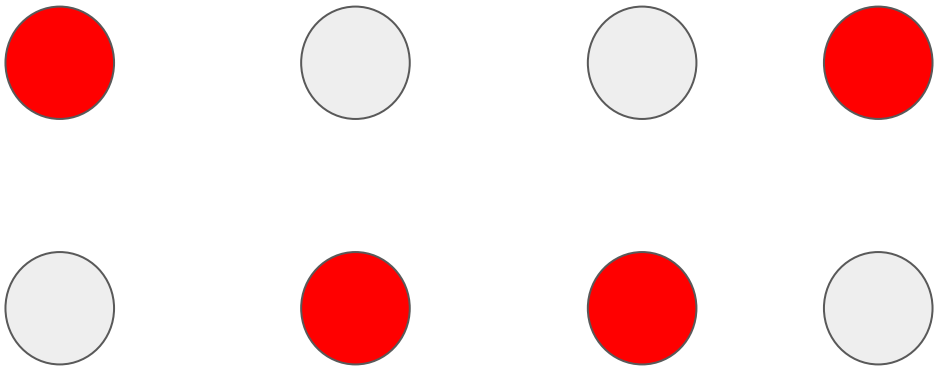
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 6

8 * 7 * 6

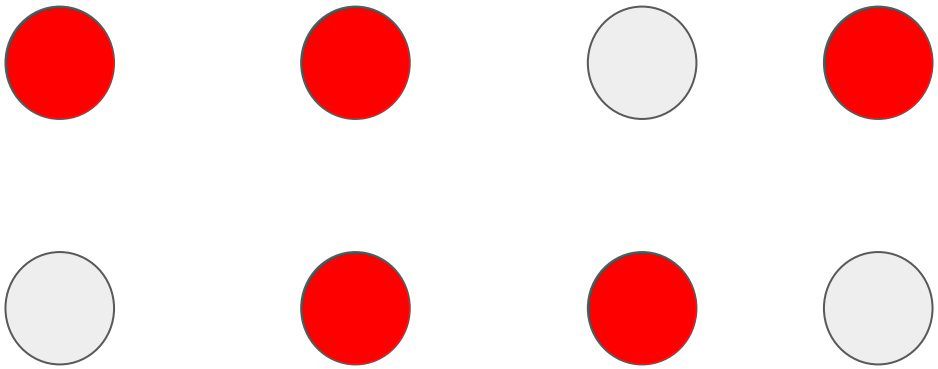
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 5

8 * 7 * 6 * 5

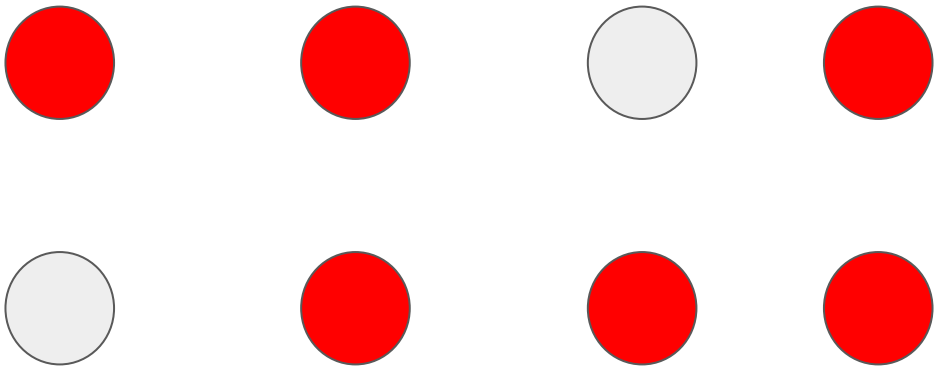
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 4

8 * 7 * 6 * 5 * 4

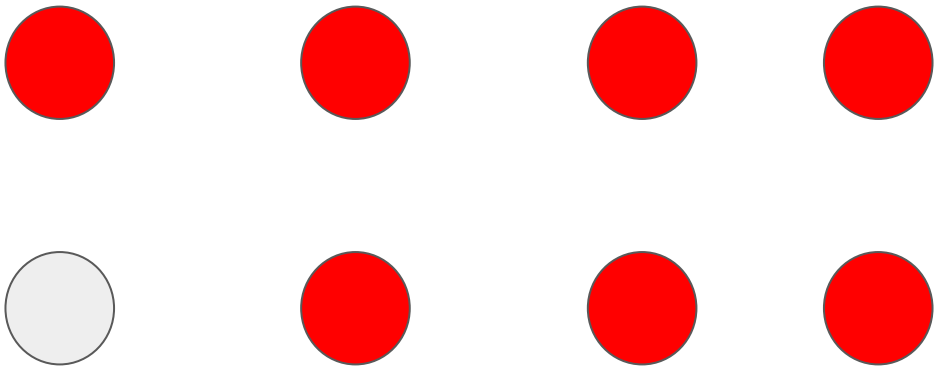
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 3

8 * 7 * 6 * 5 * 4 * 3

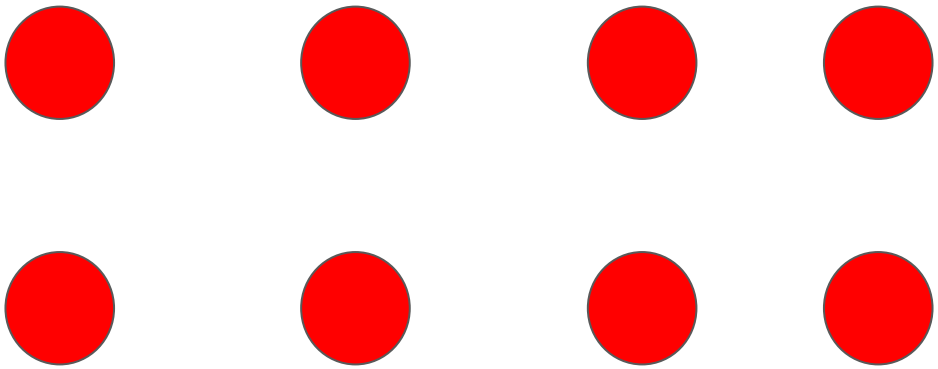
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 2

8 * 7 * 6 * 5 * 4 * 3 * 2

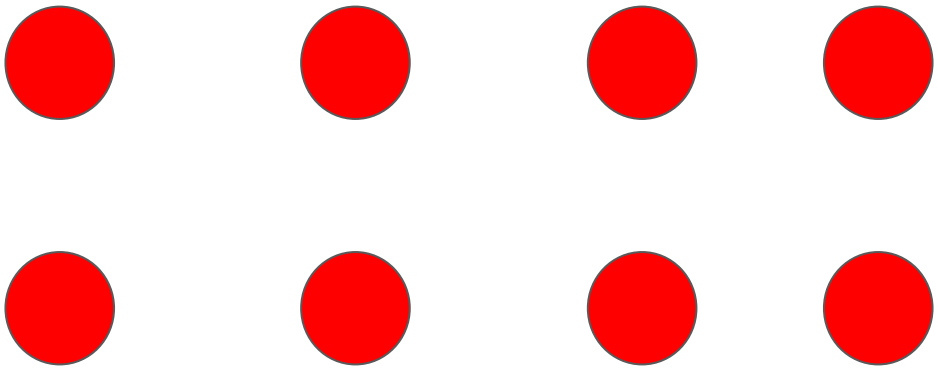
# Exercise 4 - Sweep Pattern



Randomly select 1 dot out of remaining 1

8 * 7 * 6 * 5 * 4 * 3 * 2 * 1

# Exercise 4 - Sweep Pattern



Which is

8! = 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 40.320

40.320 vs 36.500
Just a little harder!

# Question 5

1.7. You (Eve) have intercepted two ciphertexts:

$$c_1 = \texttt{111110010111100111001100000101111000011 0}$$

$$c_2 = \texttt{11111010011001111101110100001001100010 00}$$

You know that both are OTP ciphertexts, encrypted with the *same key*. You know that **either** $c_1$ is an encryption of `alpha` and $c_2$ is an encryption of `bravo` **or** $c_1$ is an encryption of `delta` and $c_2$ is an encryption of `gamma` (all converted to binary from ASCII in the standard way).

Which of these two possibilities is correct, and why? What was the key $k$?

# Question 5 - Solution

- From theory, we know that

  c = p ^ key

- From this, we obtain that

  key = p ^ c

- We know that c1 and c2 are obtained using the same key, so

  p1 ^ c1 = p2 ^ c2

- We know from the text c1 ^ c2

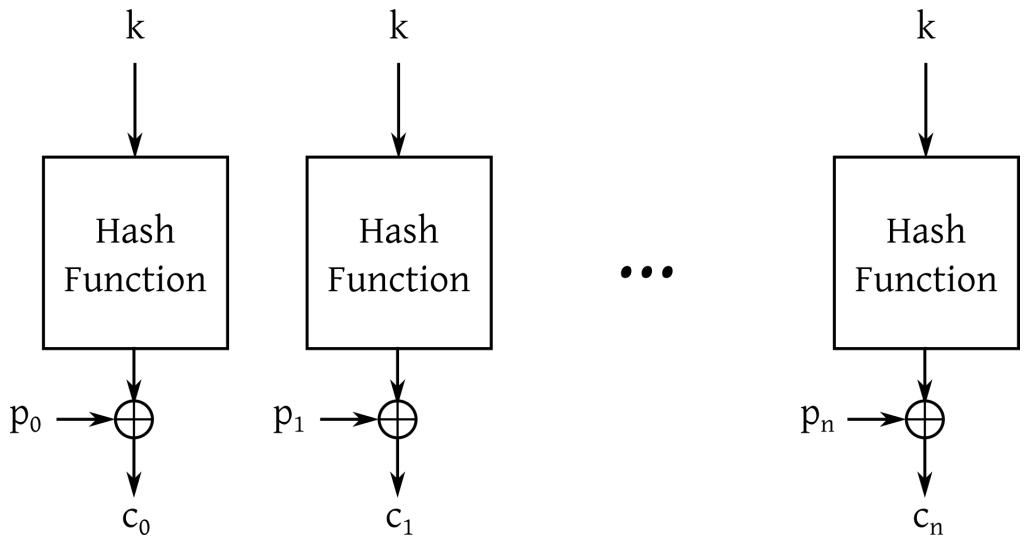
  c1 = 11111001 01111001 11001100 00010111 10000110
  c2 = 11111010 01100111 11011101 00001001 10001000

- We can see that the last byte of the two ctx is different.
  - => the last byte (character) of the two plaintexts must differ.

- We can conclude that the two plaintexts are alpha and bravo (delta and gamma have the same last character)

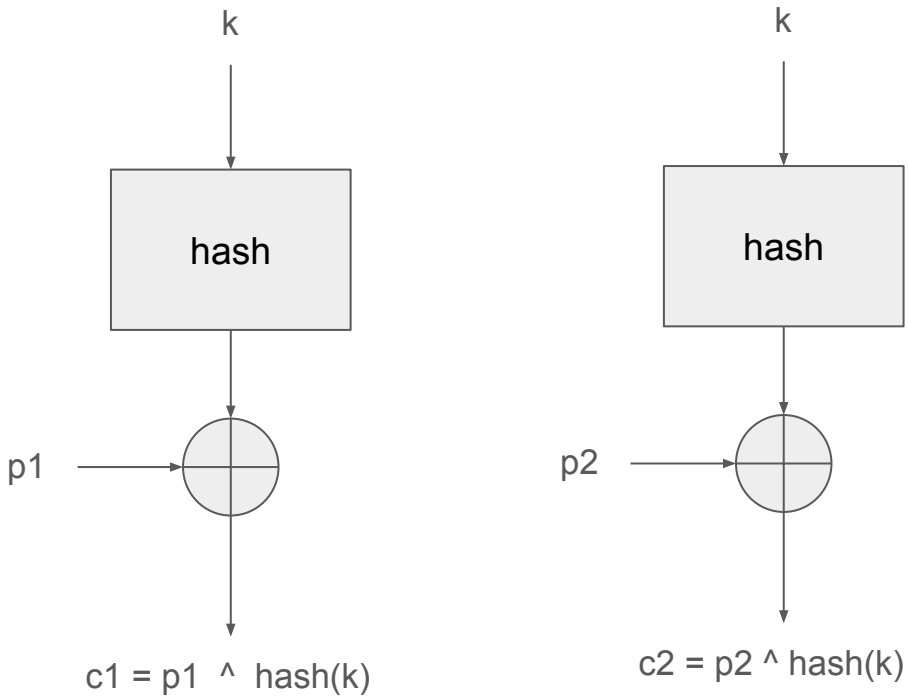
# Question 6

Consider the following construction, where k is a 1024 bit long string, known only to Alice and Bob, who want to have confidential communications. The plaintext p is split in n+1 blocks: p0,p1, … pn. Hash Function is a cryptographic hash function with a 512 bit digest. The ciphertext, which is sent on the line, is formed by the c0,c1,..., cn blocks.




Is the communication with the depicted scheme (computationally) confidential against a passive attacker, who only eavesdrops on the ciphertext? If yes, argue why; if not, describe concisely what information is leaked, and propose a countermeasure to fix the information leaked.

# Question 6 - Solution




hash(k) = c ^ p

If p1 and p2 are the same

c1 ^ p1 = c2 ^ p1
c1 = c2

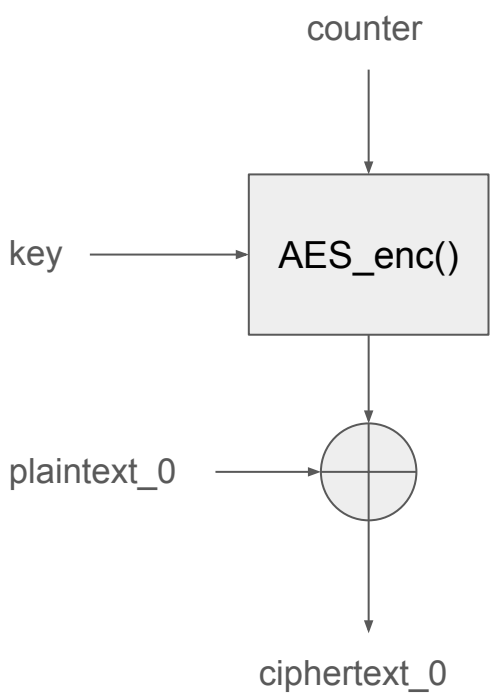
We obtain same ciphertexts!
We know that which parts of the ciphertext contains the same plaintext!

# Exercise 7

**You have the following case on your hands: one of the employees working for your company is claiming that it is not possible that files have been deleted from his hard disk without him knowing, as the disk is encrypted with AES counter mode, and no one knows the encryption key but him. Assuming that no one *actually* knows the encryption key but him, justify or contradict his claims.**

# Exercise 7

**You have the following case on your hands: one of the employees working for your company is claiming that it is not possible that files have been deleted from his hard disk without him knowing, as the disk is encrypted with AES counter mode, and no one knows the encryption key but him. Assuming that no one *actually* knows the encryption key but him, justify or contradict his claims.**




AES-CTR allows modifications of the encrypted data which cannot be detected: indeed, it is sufficient to apply the same modifications flipping bits in the ciphertext, to see them reflected in the plaintext.

# Exercise 7

**You want to employ sentences from a book as passwords, and obtain the same security level of picking a 128b random string. Consider that “The Tempest” by Shakespeare contains 26518 words, and can be optimally compressed in 53210 bytes. For simplicity assume that the information content of each word is the same. How many words from The Tempest should you pick as a password?**

# Exercise 7

**You want to employ sentences from a book as passwords, and obtain the same security level of picking a 128b random string. Consider that “The Tempest” by Shakespeare contains 26518 words, and can be optimally compressed in 53210 bytes. For simplicity assume that the information content of each word is the same. How many words from The Tempest should you pick as a password?**

- We consider each word as a single symbol
- On average, each word carries
  (53210 * 8) / 26518 ~= 16 bits
  of information
- Picking 16 random words allows to gather 128bits of information

# Exercise 8

**Consider the problem of performing periodic firmware updates to an air-gapped machine (i.e., a machine without network connection available to it). The machine is endowed with an USB port, an operating system of your choice, a timezone-synchronous reliable clock. Design a simple cryptographic protocol to validate the firmware updates, considering, as an attacker model, everything transporting the firmware from the source to the machine itself.**

# Exercise 8

**Consider the problem of performing periodic firmware updates to an air-gapped machine (i.e., a machine without network connection available to it). The machine is endowed with an USB port, an operating system of your choice, a timezone-synchronous reliable clock. Design a simple cryptographic protocol to validate the firmware updates, considering, as an attacker model, everything transporting the firmware from the source to the machine itself.**

- The manufacturer generates a secret master key of sufficient length

# Exercise 8

**Consider the problem of performing periodic firmware updates to an air-gapped machine (i.e., a machine without network connection available to it). The machine is endowed with an USB port, an operating system of your choice, a timezone-synchronous reliable clock. Design a simple cryptographic protocol to validate the firmware updates, considering, as an attacker model, everything transporting the firmware from the source to the machine itself.**

- The manufacturer generates a secret master key of sufficient length
- Fix a symmetric algorithm, and for each machine, derive a symmetric key by

  key_x = encrypt(master_key + id)

  where id is the id of the target machine

# Exercise 8

**Consider the problem of performing periodic firmware updates to an air-gapped machine (i.e., a machine without network connection available to it). The machine is endowed with an USB port, an operating system of your choice, a timezone-synchronous reliable clock. Design a simple cryptographic protocol to validate the firmware updates, considering, as an attacker model, everything transporting the firmware from the source to the machine itself.**

- The manufacturer generates a secret master key of sufficient length
- Fix a symmetric algorithm, and for each machine, derive a symmetric key by

  key_x = encrypt(master_key + id)

  where id is the id of the target machine
- Embed key_x in the filesystem of x (encrypting it)

# Exercise 8

**Consider the problem of performing periodic firmware updates to an air-gapped machine (i.e., a machine without network connection available to it). The machine is endowed with an USB port, an operating system of your choice, a timezone-synchronous reliable clock. Design a simple cryptographic protocol to validate the firmware updates, considering, as an attacker model, everything transporting the firmware from the source to the machine itself.**

- The manufacturer generates a secret master key of sufficient length
- Fix a symmetric algorithm, and for each machine, derive a symmetric key by

  key_x = encrypt(master_key + id)

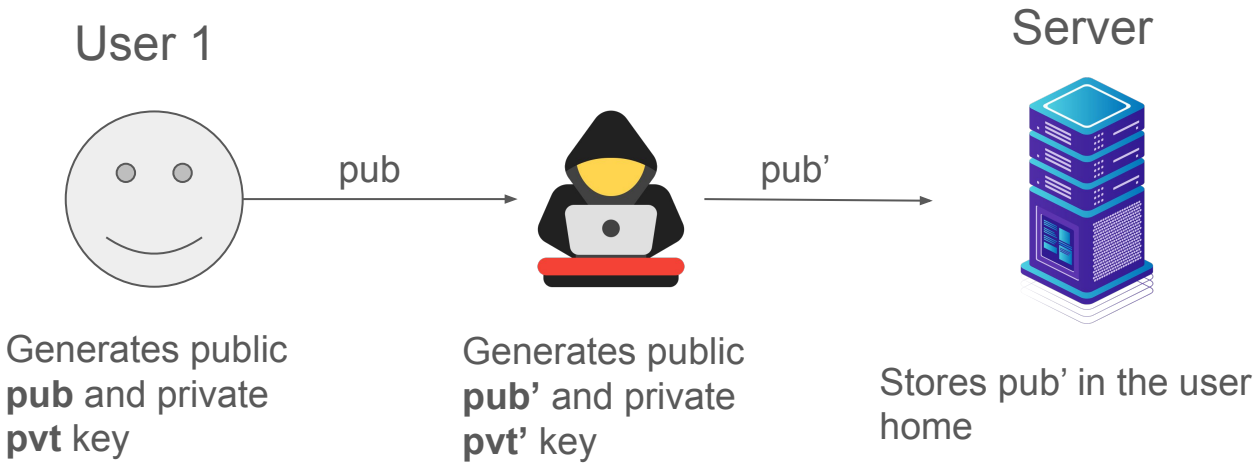
  where id is the id of the target machine
- Embed key_x in the filesystem of x (encrypting it)
- Sign each firmware using key_x!

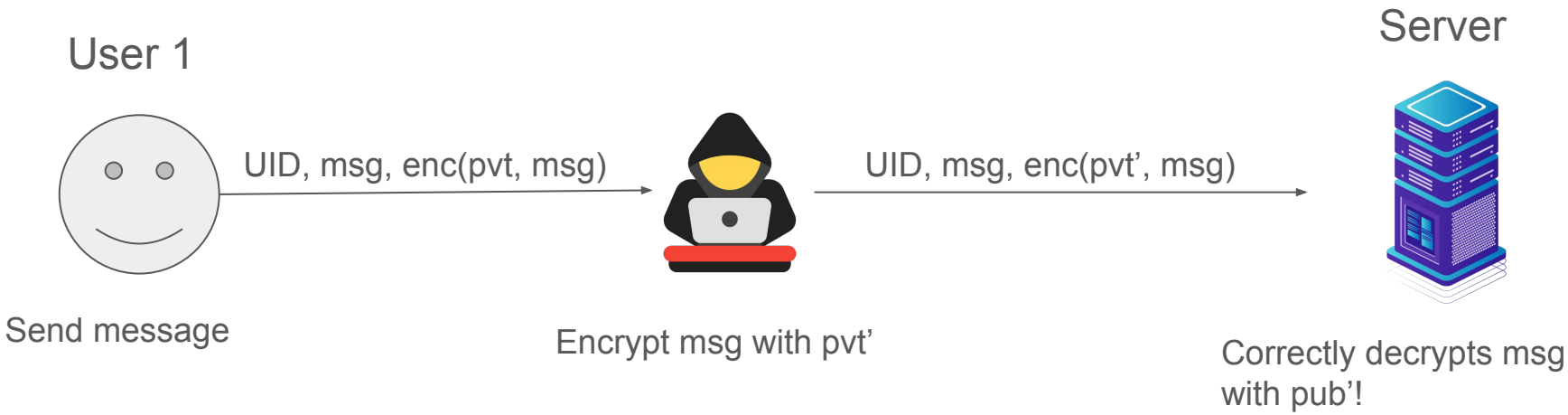
# Exercise 9

**Consider the following user authentication mechanism, implemented on a server, which uses an asymmetric encryption algorithm, for which any 1024 bit string is a valid ciphertext :**

1. **Upon user enrollment, the user generates an asymmetric encryption keypair, and sends in cleartext the public key to the server administrator. The server administrator checks reliably that the public key corresponds to the user, and it hasn't been altered in transit, and stores the (user ID, public key pair) in the home directory of the user.**
2. **Upon a login attempt, the user picks a random 1024 bit string x and decrypts it with its private key, obtaining a string s. The user sends the triple (user ID, x,s) to the server**
3. **The server employs the public key corresponding to the user ID to encrypt s, and checks if it obtains the x it received from the user. If it matches, the user is authenticated.**

# Exercise 9


User 1
Server
pub
pub'
Generates public **pub** and private **pvt** key
Generates public **pub'** and private **pvt'** key
Stores pub' in the user home


User 1
Server
UID, msg, enc(pvt, msg)
UID, msg, enc(pvt', msg)
Send message
Encrypt msg with pvt'
Correctly decrypts msg with pub'!

# Exercise 9

**Consider the usual Public Key Infrastructure (PKI) approach to certificate validation. Certificates are usually arranged in a tree, where each node is a certificate, and an edge from certificate A to certificate B states that the issuer of certificate B is the subject of certificate A.**
**What are the issues arising from changing the tree into a generic graph?**
**Can they be fixed by changing the validation process?**

The two generalisations in moving from a tree to a generic graph are:

- Nodes with more than one predecessor: they correspond to certificates with more than one issuer. This requires a change in the validation process, as a decision should be taken if only some of the signatures on the certificate are valid.
- Non trivial cycles (cycles already exist on root CAs): a non trivial cycle requires the validation process to operate loop detection (e.g., by Floyd's turtle and hare algorithm), and implies that any certificate in a cycle should be regarded as self signed. Indeed, its validity depends, cyclically, on itself.

The End

# Extra 1

2.4. Show that the following libraries are **not** interchangeable. Describe an explicit distinguishing calling program, and compute its output probabilities when linked to both libraries:

| $\mathcal{L}_{\text{left}}$ |
|---|
| $\underline{\textsc{eavesdrop}(m_L, m_R \in \{0,1\}^\lambda):}$ |
| $k \leftarrow \{0,1\}^\lambda$ |
| $c := k \oplus m_L$ |
| return $(k, c)$ |

| $\mathcal{L}_{\text{right}}$ |
|---|
| $\underline{\textsc{eavesdrop}(m_L, m_R \in \{0,1\}^\lambda):}$ |
| $k \leftarrow \{0,1\}^\lambda$ |
| $c := k \oplus m_R$ |
| return $(k, c)$ |

2.4. Show that the following libraries are **not** interchangeable. Describe an explicit distinguishing calling program, and compute its output probabilities when linked to both libraries:

| $\mathcal{L}_{\text{left}}$ |
|---|
| $\textsc{eavesdrop}(m_L, m_R \in \{0,1\}^\lambda)$: |
| $k \leftarrow \{0,1\}^\lambda$ |
| $c := k \oplus m_L$ |
| return $(k, c)$ |

| $\mathcal{L}_{\text{right}}$ |
|---|
| $\textsc{eavesdrop}(m_L, m_R \in \{0,1\}^\lambda)$: |
| $k \leftarrow \{0,1\}^\lambda$ |
| $c := k \oplus m_R$ |
| return $(k, c)$ |

```
tmp_k,tmp_c = eavesdrop(0^lambda,1^lambda)
if (tmp_k^tmp_c = 0^lambda)
   return "it's L_left!"
else
   return "it's L_right!"
```

The distinguisher works with Pr = 1

5.12. Let $G_1$ and $G_2$ be deterministic functions, each accepting inputs of length $\lambda$ and producing outputs of length $3\lambda$.

(a) Define the function $H(s_1 \| s_2) = G_1(s_1) \oplus G_2(s_2)$. Prove that if **either** of $G_1$ or $G_2$ (or both) is a secure PRG, then so is $H$.

(b) What can you say about the simpler construction $H(s) = G_1(s) \oplus G_2(s)$, when one of $G_1, G_2$ is a secure PRG?

**Definition**

A CSPRNG is a deterministic function $\text{PRNG}: \{0,1\}^\lambda \to \{0,1\}^{\lambda+l}$ whose output cannot be distinguished from an uniform random sampling of $\{0,1\}^{\lambda+l}$ in $\mathcal{O}(\text{poly}(\lambda))$. $l$ is the CSPRNG stretch.

HINTS

$\text{VERNAM}(\lambda, \texttt{ptx} \in \{0,1\}^\lambda)$

$k \xleftarrow{\$} \{0,1\}^\lambda$

$\texttt{ctx} = \texttt{ptx} \oplus k$

**return** $\texttt{ctx}$

$\text{IDEAL}(\lambda, \texttt{ptx} \in \{0,1\}^\lambda)$

$\texttt{ctx} \xleftarrow{\$} \{0,1\}^\lambda$

**return** $\texttt{ctx}$

5.12. Let $G_1$ and $G_2$ be deterministic functions, each accepting inputs of length $\lambda$ and producing outputs of length $3\lambda$.

(a) Define the function $H(s_1 \| s_2) = G_1(s_1) \oplus G_2(s_2)$. Prove that if **either** of $G_1$ or $G_2$ (or both) is a secure PRG, then so is $H$.

(b) What can you say about the simpler construction $H(s) = G_1(s) \oplus G_2(s)$, when one of $G_1, G_2$ is a secure PRG?

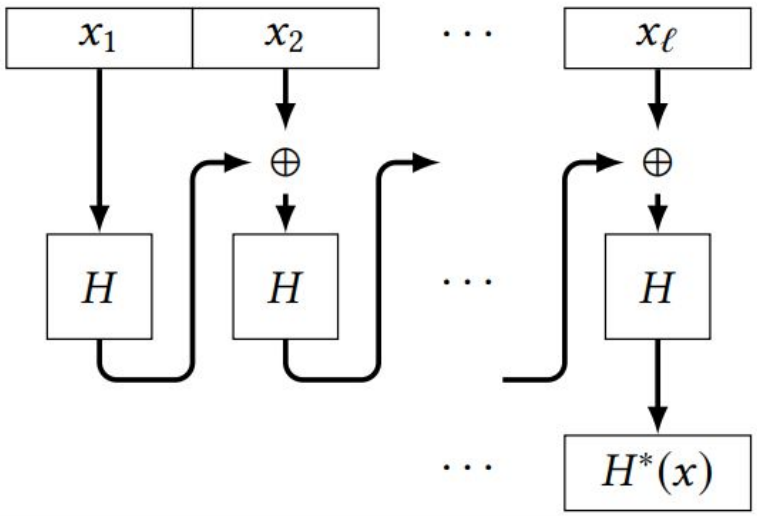
a) Assume that G_1 is the good PRG (the same reasoning applies if it's G_2). If G_1 the output of G_1 cannot be told apart from a uniformly random sampled string of 3*lambda bits, then the value G_1(s_1)^G_2(s_2) is not distinguishable from random by Shannon's theorem (it's essentially a one time pad). As a consequence, the output of H is indistinguishable from a uniformly randomly chosen string.

b) You can reduce this case to the previous one. It's just feeding the previous H function with a non uniformly sampled string.

# Extra 3

**11.13.** Let $H$ be a collision-resistant hash function with output length $n$. Let $H^*$ denote iterating $H$ in a manner similar to CBC-MAC:

$H^*(x_1 \cdots x_\ell)$:
  // *each* $x_i$ *is* $n$ *bits*
  $y_0 := 0^n$
  for $i = 1$ to $\ell$:
    $y_i := H(x_i \oplus y_{i-1})$
  return $y_i$




Show that $H^*$ is **not** collision-resistant. Describe a successful attack.

**11.13.** Let $H$ be a collision-resistant hash function with output length $n$. Let $H^*$ denote iterating $H$ in a manner similar to CBC-MAC:

$H^*(x_1 \cdots x_\ell)$:

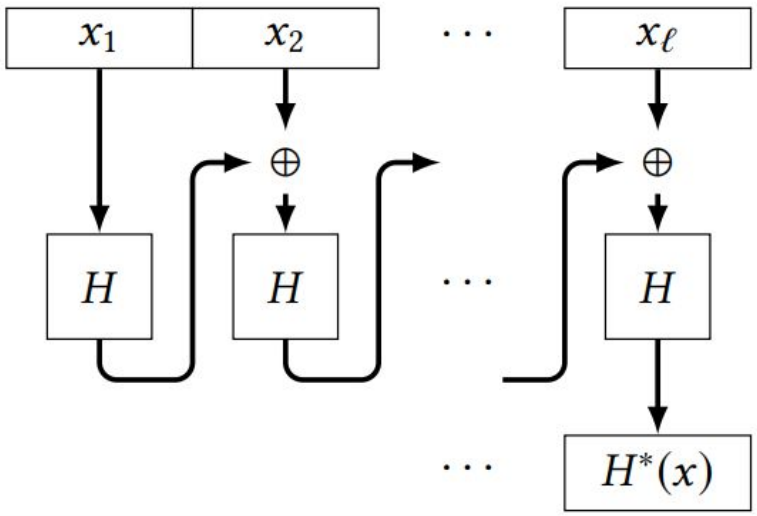
// *each* $x_i$ *is* $n$ *bits*

$y_0 := 0^n$

for $i = 1$ to $\ell$:

$\quad y_i := H(x_i \oplus y_{i-1})$

return $y_i$




Show that $H^*$ is **not** collision-resistant. Describe a successful attack.

Take a random message x=(x_1,..,x_l), compute the output of H*, call it d. Compute pad=d^x_l. H*(x||pad) will have the same hash as x since the last step will compute the hash of pad^H*(x) = d^x_l^H*(x) = x_l

# Extra 4

10.1. Consider the following MAC scheme, where $F$ is a secure PRF with $in = out = \lambda$:

| KeyGen: | $\mathsf{MAC}(k, m_1 \| \cdots \| m_\ell)$: // *each* $m_i$ *is* $\lambda$ *bits* |
|---|---|
| $k \leftarrow \{0,1\}^\lambda$ | $m^* := 0^\lambda$ |
| return $k$ | for $i = 1$ to $\ell$: |
| | $\quad m^* := m^* \oplus m_i$ |
| | return $F(k, m^*)$ |

Show that the scheme is **not** a secure MAC. Describe a distinguisher and compute its advantage.

10.1. Consider the following MAC scheme, where $F$ is a secure PRF with $in = out = \lambda$:

| KeyGen: | $\mathrm{MAC}(k, m_1 \| \cdots \| m_\ell)$: // *each* $m_i$ *is* $\lambda$ *bits* |
|---|---|
| $k \leftarrow \{0,1\}^\lambda$ | $m^* := 0^\lambda$ |
| return $k$ | for $i = 1$ to $\ell$: |
| | $\quad m^* := m^* \oplus m_i$ |
| | return $F(k, m^*)$ |

Show that the scheme is **not** a secure MAC. Describe a distinguisher and compute its advantage.

A secure MAC does not allow an attacker, who has seen a valid message-tag pair, to forge a valid tag for a different message. Assume you have a valid (m,t) pair. Denote with (m_1,...,m_l) the lambda bit long blocks of the message. Draw a lambda bits string at your leisure, call it pad.

The pair ( (m_1^pad,m_2^pad,...,m_l), t) is a valid message-tag pair, where the message differs from m according to the pad you chose.

# Extra 5

Claim 1.3 *For every $m \in \{0,1\}^\lambda$, the distribution* EAVESDROP$(m)$ *is the* ***uniform distribution*** *on $\{0,1\}^\lambda$. Hence, for all $m, m' \in \{0,1\}^\lambda$, the distributions* EAVESDROP$(m)$ *and* EAVESDROP$(m')$ *are identical.*

1.9. Suppose we modify the subroutine discussed in Claim 1.3 so that it also returns $k$:

$$\boxed{\begin{array}{l} \underline{\textsc{eavesdrop}'(m \in \{0,1\}^\lambda):} \\ \quad k \leftarrow \{0,1\}^\lambda \\ \quad c := k \oplus m \\ \quad \text{return } (k, c) \end{array}}.$$

Is it still true that for every $m$, the output of EAVESDROP$'(m)$ is distributed uniformly in $(\{0,1\}^\lambda)^2$? Or is the output distribution different for different choice of $m$?

Claim 1.3 *For every $m \in \{0,1\}^\lambda$, the distribution* EAVESDROP$(m)$ *is the* ***uniform distribution*** *on $\{0,1\}^\lambda$. Hence, for all $m, m' \in \{0,1\}^\lambda$, the distributions* EAVESDROP$(m)$ *and* EAVESDROP$(m')$ *are identical.*

1.9. Suppose we modify the subroutine discussed in Claim 1.3 so that it also returns $k$:

$$\boxed{\begin{array}{l} \underline{\textsc{eavesdrop}'(m \in \{0,1\}^\lambda):} \\ \quad k \leftarrow \{0,1\}^\lambda \\ \quad c := k \oplus m \\ \quad \text{return } (k, c) \end{array}}.$$

Is it still true that for every $m$, the output of EAVESDROP$'(m)$ is distributed uniformly in $(\{0,1\}^\lambda)^2$? Or is the output distribution different for different choice of $m$?

It's false. Simply consider the value obtained xor-ing the two returned values: it's k^c = m. It's clear that the distribution of m does not look like a random independent bitstring with respect to m, indeed, it's the message!